

第5章 问题求解

第10讲 典型案例分析

邹华

武汉大学计算机学院

第5章 问题求解

2

- 一、问题求解过程与框架
- 二、算法策略概述
- 三、可求解与难求解问题
- 四、问题求解案例实训

可求解与难求解问题

3

用计算复杂性划分问题

【P类问题】 多项式问题(Polynomial Problem), 即: 可以找出一个呈现 $O(n^a)$ 复杂性算法**求出精确解**的问题, 其中 a 为常数。P类问题是指计算机可以在有限时间内能求出精确解的问题,

【NP类问题】 非确定性多项式问题(Non-deterministic Polynomial), 即: 可以找出一个呈现 $O(n^a)$ 复杂性算法来验证一个可能解是否是正确的问题。

✓有些问题, 其答案无法直接计算得到, 但可通过间接的猜算/试算来得到, 这就是非确定性问题(Non-deterministic)。

✓虽然在多项式时间内难于求解, 但给定一个解却不难在多项式时间内验证其正确性的问题, 即是NP类问题。

【NPC类问题】 完全非确定性多项式问题(NP-Complete), 即: (1)如果NP问题的所有可能解都可以在多项式时间内进行验算, (2)求精确解需要遍历所有可能解, (3)遍历所有可能解的复杂度是指数或阶乘级别的; 则为NP-Complete问题。

- 一个NP问题, 并非其每个解都可以在多项式时间内验算, 即: 有些解是可以验算的, 而有些解是不能验算的。
- 如果一个NP问题其需要验算的解空间是用非多项式函数(指数函数或阶乘函数)来表达的, 则是NP-Complete问题。
- 如果一个NP问题经过约简后得到的问题仍旧是NP问题, 则是NP-Complete问题。

问: 加密算法应该设计成一个什么问题呢?

注意: 这里给出的是易于理解的表述。严格的定义需要学习计算理论课程!

可求解与难求解问题

可解性问题与难解性问题

P类问题

(多项式时间内能求精确解)

NP类问题

(多项式时间内能验证一个解)

NPC类问题

(1)多项式时间内能验证每一个解,(2)求精确解需要遍历解空间,(3)指数时间遍历解空间

NP-Hard/NP难问题

难解性问题

- 非NP问题
- 多项式时间内不一定能验证一个解

可求解与难求解问题

5

NPC类问题如何求解

TSP问题是
NPC类问题

NPC类问题

TSP问题的
遍历算法

穷举法或称遍历法：对解空间中的每一个可能解进行验证，直到所有的解都被验证是否正确，便能得到精确的结果---精确解

可能是 $O(n!)$ 或 $O(a^n)$

TSP问题的
贪心算法
仿生学算法

近似解求解算法---近似解

应该是 $O(n^a)$

$\Delta = | \text{近似解} - \text{精确解} |$
满意解： Δ 充分小时的近似解

进化算法, 遗传算法, 蚁群算法, 蜂群算法, ...

测练题： 关于P类问题，下列说法正确的是。

A

P类问题是可以在有限时间内求解的问题

B

P类问题是能够用多项式方法求解的问题

C

P类问题是能够用多项式方法求解且其复杂性为多项式的问题

D

P类问题是指这样的问题，即能够找出求解该问题的一个算法，且该算法的时间复杂性为多项式

E

P类问题是指这样的问题，即能够找出求解该问题的一个算法，且该算法与某个多项式有关

提交

测练题：关于NP类问题，下列说法正确的是_____。

- ☐ A NP类问题是不能在有限时间内求解的问题
- ☐ B NP类问题是不能够用多项式方法求解的问题
- ☐ C NP类问题是不能够用多项式方法求解且其复杂性不为多项式的问题
- ☒ D NP类问题是指不确定其能否在多项式时间内求解，但能够在多项式时间内判定一个解是否是问题的解的问题
- ☐ E NP类问题是指既能够在多项式时间内判定，又能够在多项式时间内求解的问题

提交

测练题：关于NPC类问题，下列说法正确的是_____。

A

NPC类问题是最难求解的NP类问题

B

NPC类问题，是目前已知其精确解求解算法都是指数级算法的问题

C

NPC类问题是不可求解的问题

D

NPC类问题是难以在有限时间内求解的问题

E

NPC类问题是当问题规模很大时难以在有限时间内求解的问题

提交

测练题：关于P类问题和NP类问题，下列说法正确的是_____。

☐ A

P类问题是能在有限时间内求解的问题，NP类问题是不能在有限时间内求解的问题

☒ B

如果一个问题为P类问题，则该问题也是NP类问题

☐ C

如果一个问题为P类问题，则该问题不是NP类问题

☐ D

如果一个问题为NP类问题，则该问题也是P类问题

☒ E

如果一个问题为NP类问题，则该问题也可能是P类问题

提交

第5章 问题求解

10

- 一、问题求解过程与框架
- 二、算法策略概述
- 三、可求解与难求解问题
- 四、问题求解案例实训

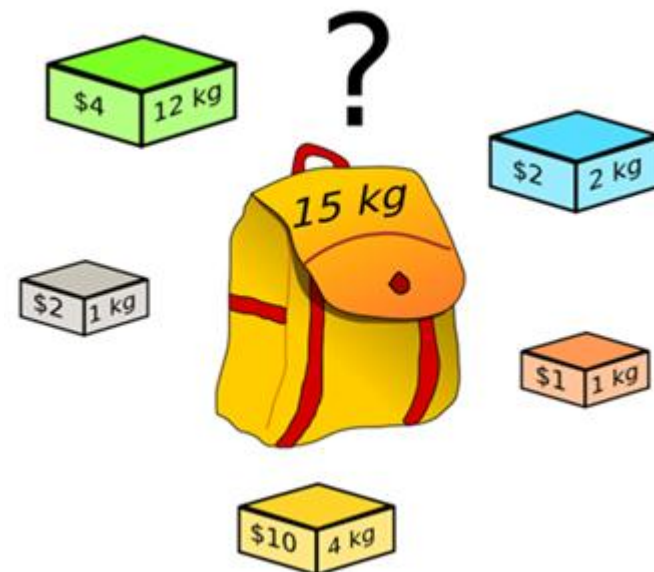
TSP问题求解案例

11

背包问题求解-遍历策略【练习】

0-1背包问题：有 n 种物品，物品 j 的重量为 w_j ，价格为 p_j 。假定所有物品的重量和价格都是非负的，背包所能承受的最大重量为 W 。如果限定每种物品只能选择0个或1个，则问题称为0-1背包问题。

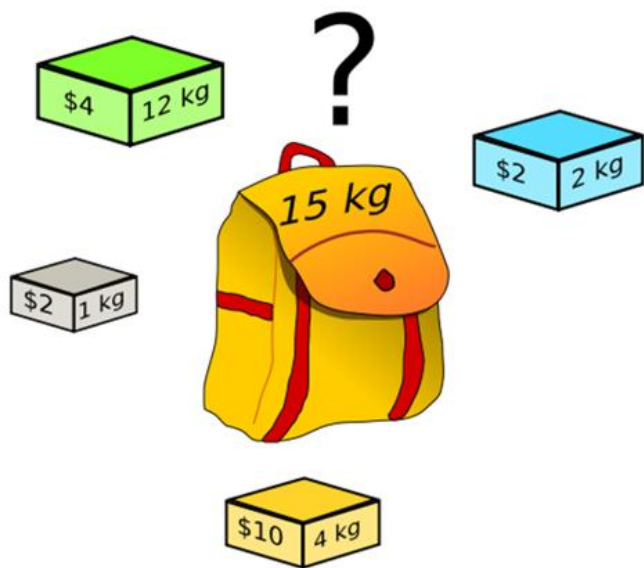
	股票投资组合问题	产品品种的组合优化选择问题	背包问题(0-1 背包)	背包问题（有界背包）
问题 简要表述	有 n 种股票可供选择，每种股票 j 最多买 b_j 股，怎样一个组合，才能使股票收益最大。	有 n 种产品可供销售，怎样一种组合，才能使销售价值最大。	有 n 种物品，每种物品最多 1 件，怎样选择物品放入背包，才能使背包中物品价值最大	有 n 种物品，每种物品 j 最多选择 b_j 个，怎样选择物品放入背包，才能使背包中物品价值最大



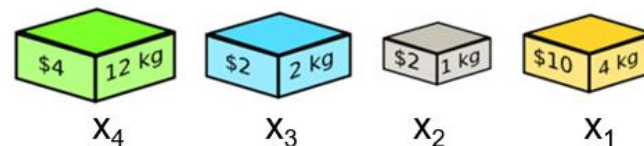
TSP问题求解案例

12

背包问题求解-遍历策略【练习】



(a)背包问题示意



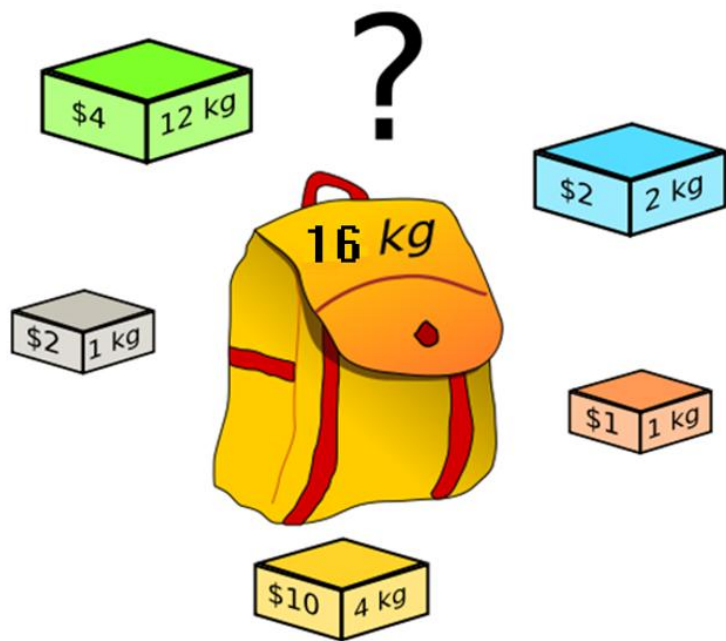
可能解空间 $X_4X_3X_2X_1$	$\sum_{j=1}^{k=n} w_j x_j \leq W$ 被选中物品的总重量	可行解空间 $X_4X_3X_2X_1$	$\sum_{j=1}^{k=n} p_j x_j$ 被选中物品的总价值	问题的最优解 (精确解) $X_4X_3X_2X_1$
0000	0, 满足约束	0000	0	
0001	4, 满足约束	0001	10	
0010	1, 满足约束	0010	2	
0011	5, 满足约束	0011	12	
0100	2, 满足约束	0100	2	
0101	2, 满足约束	0101	12	
0110	3, 满足约束	0110	4	
0111	7, 满足约束	0111	14	0111
1000	12, 满足约束	1000	4	
1001	16, 不满足约束			
1010	13, 满足约束	1010	6	
1011	17, 不满足约束			
1100	14, 满足约束	1100	6	
1101	18, 不满足约束			
1110	15, 满足约束	1110	8	
1111	19, 不满足约束			

(b)背包问题求解过程示意

TSP问题求解案例

13

背包问题求解-贪心策略【练习】



(a)背包问题示意

						
X ₅	X ₄	X ₃	X ₂	X ₁		
物品选择次序	策略 1: 重量小的物品 优先	策略 1: 被选中的重量 和价值	策略 2: 价值高的物品 优先	策略 2: 被选中的重量 和价值	策略 3: 性价比高的物品 优先	策略 3: 被选中的重量 和价值
第 1 次	X ₅	1,1	X ₁	4,10	X ₁	4,10
第 2 次	X ₂	2,3	X ₄	16,14	X ₂	5,12
第 3 次	X ₃	4,5			X ₃	7,14
第 4 次	X ₁	8,15			X ₅	8,15
最终结果	X ₅ X ₄ X ₃ X ₂ X ₁ =10111	8, 15	X ₅ X ₄ X ₃ X ₂ X ₁ =01001	16, 14	X ₅ X ₄ X ₃ X ₂ X ₁ =10111	8, 15

(b)不同贪心策略求解背包问题示意图

TSP问题求解案例

14

背包问题求解-贪心策略【练习】

	股票投资组合问题	产品品种的组合优化选择问题	背包问题(0-1 背包)	背包问题(有界背包)
问题 简要表述	有 n 种股票可供选择, 每种股票 j 最多买 b_j 股, 怎样一个组合, 才能使股票收益最大。	有 n 种产品可供销售, 怎样一种组合, 才能使销售价值最大。	有 n 种物品, 每种物品最多 1 件, 怎样选择物品放入背包, 才能使背包中物品价值最大	有 n 种物品, 每种物品 j 最多选择 b_j 个, 怎样选择物品放入背包, 才能使背包中物品价值最大
“输入/已知” 的抽象表示	n 种股票可用编号 $1, \dots, n$ 表示	n 种产品可用编号 $1, \dots, n$ 表示	n 种物品可用编号 $1, \dots, n$ 表示	n 种物品可用编号 $1, \dots, n$ 表示
“输入/已知” 的抽象表示	每种股票 j 的价格为 w_j 。	每种产品 j 的库存数量为 w_j 。(或者每种产品 j 的生产能力为 w_j)	每种物品 j 的重量为 w_j 。	每种物品 j 的重量为 w_j 。
“输入/已知” 的抽象表示	每种股票 j 的收益为 p_j 。	每种产品 j 的价格为 p_j 。	每种物品 j 的价值为 p_j 。	每种物品 j 的价值为 p_j 。
“输入/已知” 的抽象表示	可用于购买股票的资本为 W	市场可接受产品的总数量 W 。(或者企业能生产产品的总数量 W)	背包的总重量为 W	背包的总重量为 W
“输出/结果” 的抽象表示	求一组合 x , $x=(x_1, x_2, x_3, \dots, x_n)$. 其中股票 j 选择 x_j 股。 $x_j=0, \dots, b_j$ 为购买股数	求一组合 x , $x=(x_1, x_2, x_3, \dots, x_n)$. 其中产品 j 选择 x_j 个。 $x_j=0, \dots, b_j$ 为销售数量	求一组合 x , $x=(x_1, x_2, x_3, \dots, x_n)$. 其中 $x_j=0$ 不选物品 j , $=1$ 选择物品 j 。	求一组合 x , $x=(x_1, x_2, x_3, \dots, x_n)$. 其中物品 j 选择 x_j 个。 $x_j=0, \dots, b_j$ 为选择个数
“输出/结果” 需满足约束	$\sum_{j=1}^n w_j x_j \leq W$, 其中 $x_j=0, \dots, b_j$	$\sum_{j=1}^n w_j x_j \leq W$, 其中 $x_j=0, \dots, b_j$	$\sum_{j=1}^n w_j x_j \leq W$, 其中 $x_j=0, 1$	$\sum_{j=1}^n w_j x_j \leq W$, 其中 $x_j=0, \dots, b_j$
“输出/结果” 需优化达到的目标	x 要使得 $\sum_{j=1}^n p_j x_j$ 最大	x 要使得 $\sum_{j=1}^n p_j x_j$ 最大	x 要使得 $\sum_{j=1}^n p_j x_j$ 最大	x 要使得 $\sum_{j=1}^n p_j x_j$ 最大

TSP问题：算法的数据结构设计

问题求解相关数据如何存储？

城市映射为编号: A---1, B---2, C---3, D---4

城市间距离关系: 表或二维数组D, 用 $D[i][j]$ 或 $D[i,j]$ 来确定欲处理的每一个元素

城市编号	1	2	3	4
1		2	6	5
2	2		4	4
3	6	4		2
4	5	4	2	

D

$D[2][3]$

访问路径/解: 一维数组S, 用 $S[j]$ 来确定每一个元素

S	1	4	3	2
	S[1]	S[2]	S[3]	S[4]

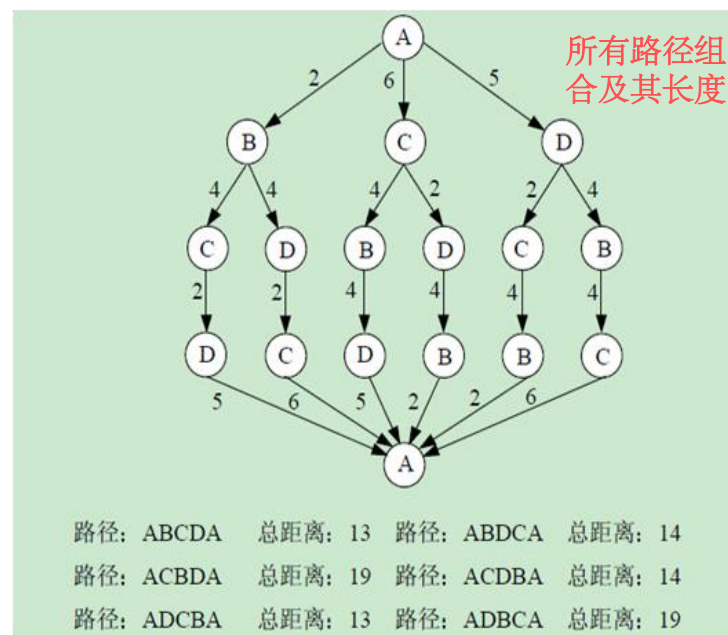
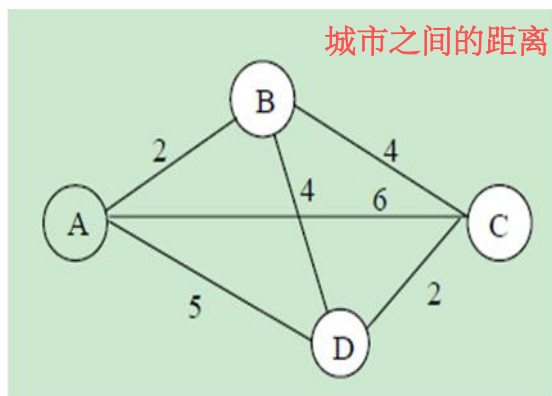
{A->D->C->B->A}

问题求解的进一步思考

16

蛮干/遍历是基本的算法策略

遍历算法求解TSP问题：列出每一条可供选择的路线，计算出每条路线的总里程，最后从中选出一条最短的路线。



TSP问题：算法的控制结构设计

求解TSP问题的遍历算法的表达

	S[1]	S[2]	S[3]	S[4]
S	1	2	3	4
	1	2	4	3
	1	3	2	4
	1	3	4	2
				

$$\text{Distance} = D[S[4]][S[1]] + \sum_{i=1 \text{ to } 3} D[S[i]][S[i+1]]$$

当前最短路径 MS

1	2	3	4
---	---	---	---

当前最短距离 DTemp

D	城市编号	1	2	3	4
	1		2	6	5
	2	2		4	4
	3	6	4		2
	4	5	4	2	

将思想/策略
转变为“步骤”

开始

当前最短路径MS设为空，当前最短距离DTemp设为最大值

组合一条新路径S并计算该路径的距离Distance

比当前最短距离小吗？

否

是

将该路径记录为当前最短路径，并用其距离值更新当前最短距离

所有路径组合完毕否？

否

是

输出当前最短路径及当前最短距离

结束

TSP问题：算法的控制结构设计

求解TSP问题的贪心算法的表达

✓依次访问过的城市编号被记录在 $S[1], S[2], \dots, S[N]$ 中,即第 l 次访问的城市记录在 $S[l]$ 中。

✓Step(1): 从1号城市开始访问, 将城市编号1赋值给 $S[1]$ 。

✓Step(6): 第 l 次访问的城市为城市 j , 其距第 $l-1$ 次访问的城市的距离最短。

	$S[1]$	$S[2]$	$S[3]$	$S[4]$
S	1			

$l = 2, 3, 4$ ---当前要找第几个

$j = 1, 2, 3, 4$ ---城市号

D	城市编号	1	2	3	4
	1		2	6	5
	2	2		4	4
	3	6	4		2
	4	5	4	2	

Start of the Algorithm

- (1) $S[1]=1$;
- (2) $Sum=0$;
- (3) 初始化距离数组 $D[N, N]$;
- (4) $l=2$;
- (5) 从所有未访问过的城市中查找距离 $S[l-1]$ 最近的城市 j ;
- (6) $S[l]=j$;
- (7) $l=l+1$;
- (8) $Sum=Sum+Dtemp$;
- (9) 如果 $l \leq N$, 转步骤(5), 否则, 转步骤(10);
- (10) $Sum=Sum+D[1, j]$;
- (11) 逐个输出 $S[N]$ 中的全部元素;
- (12) 输出 Sum 。

End of the Algorithm

TSP问题：算法的控制结构设计

求解TSP问题的贪心算法的表达

前述第5步“从所有未访问过的城市中查找距离 $S[I-1]$ 最近的城市 j ”还是不够明确，需要进一步细化

	S[1]	S[2]	S[3]	S[4]
S	1			

$I = 2, 3, 4$ ---当前要找第几个

$L = 1, 2, \dots, I-1$ ---从第1个访问过的，到第 $I-1$ 个访问过的

$K = 2, 3, 4$ ---城市号

(5.1) $K=2$;

(5.2)将Dtemp设为一个大数(比所有两个城市之间的距离都大)

(5.3) $L=1$;

(5.4)如果 $S[L]==K$ ，转步骤5.8; //该城市已出现过，跳过

(5.5) $L=L+1$;

(5.6)如果 $L < I$ ，转5.4;

(5.7)如果 $D[K, S[I-1]] < Dtemp$ ，则 $j=K$; $Dtemp = D[K, S[I-1]]$;

(5.8) $K=K+1$;

(5.9)如果 $K \leq N$ ，转步骤5.3。

TSP问题：算法的控制结构设计

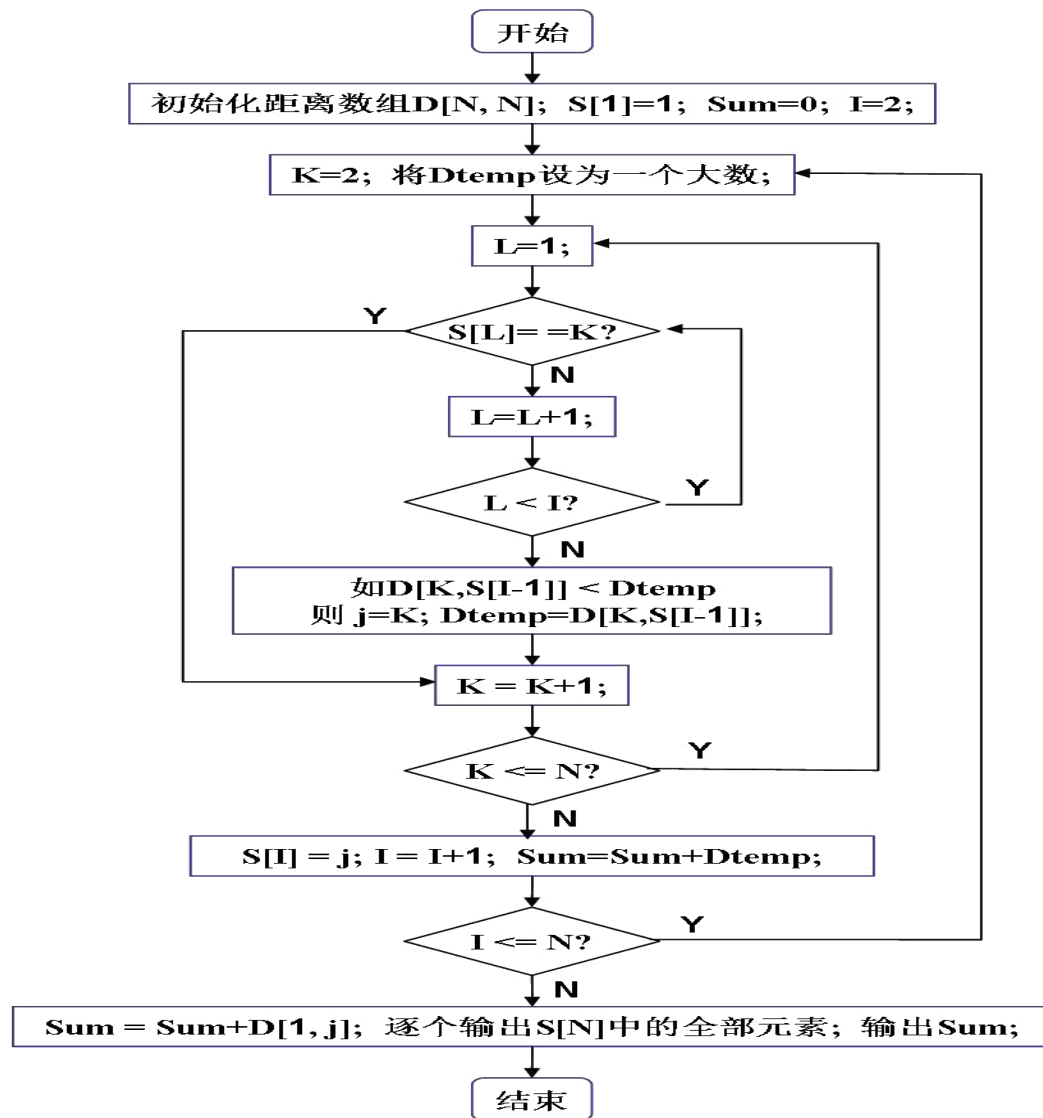
求解TSP问题的贪心算法的表达

	S[1]	S[2]	S[3]	S[4]
S	1			

$I = 2, 3, 4$ ---当前要找第几个

$K = 2, 3, 4$ ---城市号

$L = 1, 2, \dots, I-1$ ---从第1个访问过的，到第I-1个访问过的



TSP问题：算法的控制结构设计

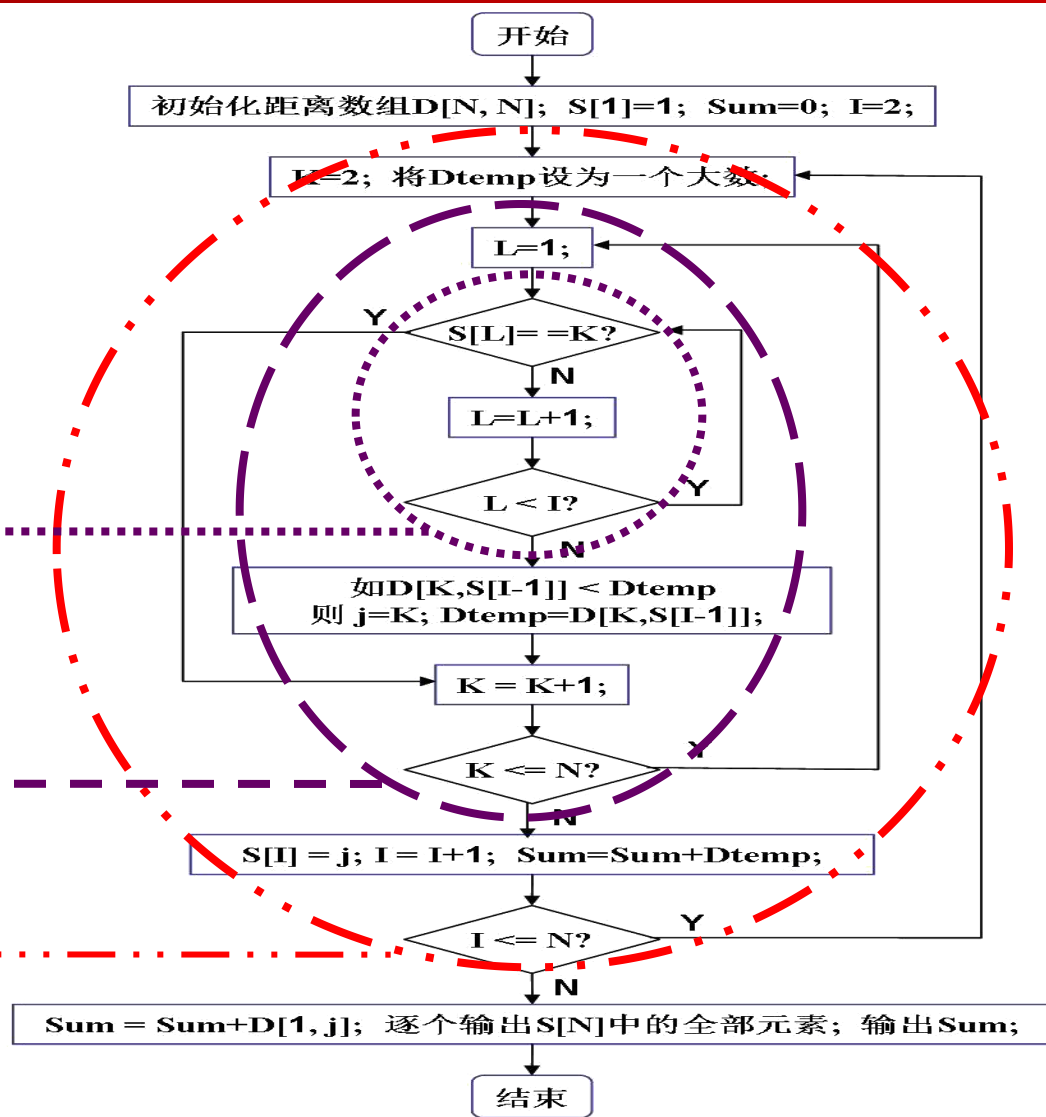
阅读并理解算法的能力

◆ 计算学科的学生不仅能够设计算法，而且会描述和表达算法，更要能读懂算法。

内层循环， L 从1至 $I-1$ ，循环判断第 K 号城市是否是已访问过的城市，如是则不参加最小距离的比较；

中层循环， K 从2号城市至 N 号城市循环，判断 $D[K, S[I-1]]$ 是否是最小值， j 记录了最小距离的城市号 K 。

外层循环， I 从2至 N 循环； $I-1$ 个城市已访问过，正在找与第 $I-1$ 个城市最近距离的城市；已访问过的城市号存储在 $S[]$ 中。



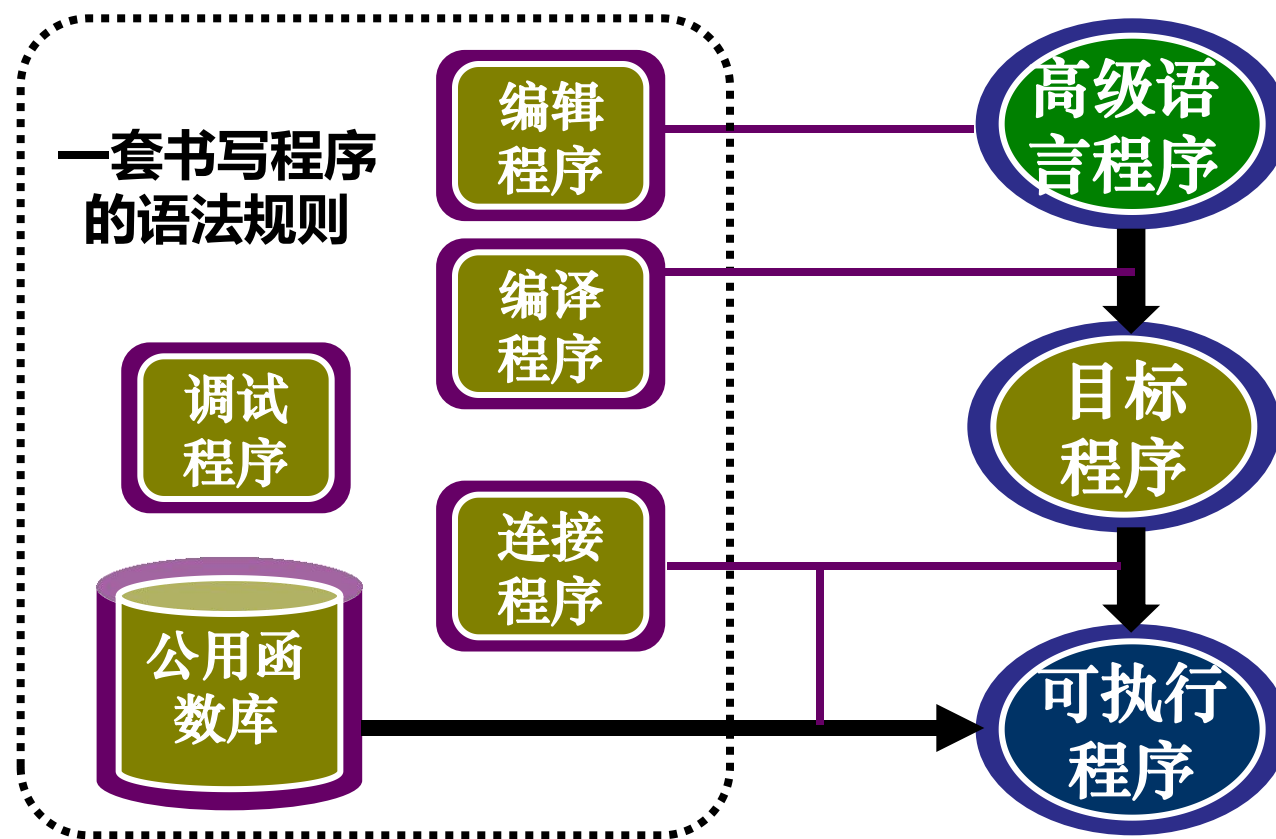
TSP问题：程序设计

22

算法的实现环境与过程

程序设计过程：编辑源程序→编译→链接→执行。

计算机语言程序
设计环境：编辑、
编译、连接、调
试、运行一体化
平台



TSP问题：程序设计

23

实现算法的程序：阅读与理解

```
main() {  判断城市K是否是已访问过的城市
```

```
...
```

```
//前面已为 K 和 I 赋过值
```

```
L=0; Found=0;
```

```
do {
```

```
    if(S[L]==K)
```

```
        { Found=1; break; }
```

```
    else L=L+1;
```

```
    } while(L<I);    //L从1到I循环
```

```
if Found==0 { //城市K未出现过 }
```

```
else { //城市K出现过 }
```

```
...
```

```
}
```

- K从1,...,n-1是城市的编号
- I是至今已访问过的城市数
- S[0]至S[I-1]中存储的是已访问过的城市的编号

S	1	4	3	2
	S[0]	S[1]	S[2]	S[3]

TSP问题：程序设计

24

实现算法的程序：阅读与理解

```
main() { ... 寻找下一个未访问过的城市j,且其距当前访问城市S[I-1]距离最短
    K=1; Dtemp=10000;
    do{ // K从1到n-1循环
        L=0; Found=0;
        do{
            if(S[L]==K)
                { Found=1; break; }
            else L=L+1;
        } while(L<I); //L从1到I循环
        if (Found==0 && D[K][S[I-1]]<Dtemp)
            { j=K; Dtemp=D[K][S[I-1]]; }
        K=K+1;
    } while(K<n); //K从1到n-1循环
    S[I]=j; Sum=Sum+Dtemp;
    ... }
```

- K从1,...,n-1是城市的编号
- I是至今已访问过的城市数
- S[I-1]中存储的是当前访问过的城市编号，要找第I个程序

S	1	4	3	2
	S[0]	S[1]	S[2]	S[3]

- D[K][S[I-1]]为城市K距当前访问过的城市的距离

TSP问题：程序设计

25

实现算法的程序：阅读与理解

TSP问题的贪心算法程序实例

```
#include <stdio.h>
#define n 4
main(){
    int D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
    S[0]=0; Sum=0; D[0][1]=2; D[0][2]=6; D[0][3]=5; D[1][0]=2; D[1][2]=4;
    D[1][3]=4; D[2][0]=6; D[2][1]=4; D[2][3]=2; D[3][0]=5; D[3][1]=4; D[3][2]=2;
    I=1; //注意程序是从 0 开始，流程图是从 1 开始的，下同。
    do { //I 从 1 到 n-1 循环
        K=1; Dtemp=10000;
        do{ //K 从 1 到 n-1 循环
            L=0; Found=0;
            do{ //L 从 1 到 I 循环
                if(S[L]==K)
                { Found=1; break; }
                else L=L+1;
            } while(L<I); //L 从 1 到 I 循环
            if(Found==0 && D[K][S[I-1]]<Dtemp)
            { j=K; Dtemp=D[K][S[I-1]]; }
            K=K+1;
        } while(K<n); //K 从 1 到 n-1 循环
        S[I]=j; I=I+1; Sum=Sum+Dtemp;
    } while(I<n); //I 从 1 到 n-1 循环
    Sum=Sum+D[1][j];
    for(j=0;j<n;j++){ printf("%d,",S[j]); } //输出城市编号序列
    printf("\n"); //换行
    printf("Total Length:%d",Sum); //输出总距离
}
```

TSP问题：算法的正确性分析

TSP问题贪心算法的正确性评价

直观上只需检查算法的输出结果中，**每个城市出现且仅出现一次**，该结果即是TSP问题的可行解，说明算法正确地求解了这些问题实例

	S[1]	S[2]	S[3]	S[4]
S	1	2	3	4

```
#include <stdio.h>
#define n 4
main()
{
    int D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
    S[0]=0; Sum=0; D[0][1]=2; D[0][2]=6; D[0][3]=5; D[1][0]=2; D[1][2]=4;
    D[1][3]=4; D[2][0]=6; D[2][1]=4; D[2][3]=2; D[3][0]=5; D[3][1]=4; D[3][2]=2;
    I=1; //注意程序是从0开始，流程图是从1开始的，下同。
    do { //I 从 1 到 n-1 循环
        K=1; Dtemp=10000;
        do { //K 从 1 到 n-1 循环
            L=0; Found=0;
            do { //L 从 0 到 I-1 循环
                if(S[L]==K)
                { Found=1; break; }
                else L=L+1;
            } while(L<I); //L 从 0 到 I-1 循环
            if(Found==0 && D[K][S[I-1]]<Dtemp)
            { j=K; Dtemp=D[K][S[I-1]]; }
            K=K+1;
        } while(K<n); //K 从 1 到 n-1 循环
        S[I]=j; I=I+1; Sum=Sum+Dtemp;
    } while(I<n); //I 从 1 到 n-1 循环
    Sum=Sum+D[j][0];
    for(j=0;j<n;j++){ printf("%d,",S[j]); } //输出城市编号序列
    printf("\n"); //换行
    printf("Total Length:%d",Sum); //输出总距离
}
```

TSP问题：算法的正确性分析

TSP问题贪心算法的效果评价

- ✓如果实例的最优解已知（问题规模小或问题已被成功求解），利用统计方法对若干问题实例的算法结果与最优解进行对比分析，即可对其进行效果评价；
- ✓对于较大规模的问题实例，其最优解往往是未知的，因此，算法的效果评价只能借助于与**前人算法**结果的比较。

	S[1]	S[2]	S[3]	S[4]
S	1	2	3	4

```
#include <stdio.h>
#define n 4
main(){
    int D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
    S[0]=0; Sum=0; D[0][1]=2; D[0][2]=6; D[0][3]=5; D[1][0]=2; D[1][2]=4;
    D[1][3]=4; D[2][0]=6; D[2][1]=4; D[2][3]=2; D[3][0]=5; D[3][1]=4; D[3][2]=2;
    I=1; //注意程序是从 0 开始，流程图是从 1 开始的，下同。
    do { //I 从 1 到 n-1 循环
        K=1; Dtemp=10000;
        do { //K 从 1 到 n-1 循环
            L=0; Found=0;
            do { //L 从 0 到 I-1 循环
                if(S[L]==K)
                { Found=1; break; }
                else L=L+1;
            } while(L<I); //L 从 0 到 I-1 循环
            if(Found==0 && D[K][S[I-1]]<Dtemp)
            { j=K; Dtemp=D[K][S[I-1]]; }
            K=K+1;
        } while(K<n); //K 从 1 到 n-1 循环
        S[I]=j; I=I+1; Sum=Sum+Dtemp;
    } while(I<n); //I 从 1 到 n-1 循环
    Sum=Sum+D[j][0];
    for(j=0;j<n;j++){ printf("%d,",S[j]); } //输出城市编号序列
    printf("\n"); //换行
    printf("Total Length:%d",Sum); //输出总距离
}
```

TSP问题：算法的复杂性分析

TSP问题贪心算法的复杂性分析

- ✓ 一个关于 n 的三重循环。
- ✓ 时间复杂度是 $O(n^3)$ 。

```
#include <stdio.h>
#define n 4
main()
{
    int D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
    S[0]=0; Sum=0; D[0][1]=2; D[0][2]=6; D[0][3]=5; D[1][0]=2; D[1][2]=4;
    D[1][3]=4; D[2][0]=6; D[2][1]=4; D[2][3]=2; D[3][0]=5; D[3][1]=4; D[3][2]=2;
    I=1; //注意程序是从0开始，流程图是从1开始的，下同。
    do { //I 从 1 到 n-1 循环——将被执行 n-1 次，简记约 n 次
        K=1; Dtemp=10000;
        do { //K 从 1 到 n-1 循环——将被执行 (n-1)*(n-1) 次，简记约 n^2 次
            L=0; Found=0;
            do { //L 从 1 到 I 循环——将被执行，简记约 n^3 次
                if(S[L]==K)
                { Found=1; break; }
                else L++;
            } while(L<I); //L 从 1 到 I 循环
            if(Found==0 && D[K][S[I-1]]<Dtemp)
            { j=K; Dtemp=D[K][S[I-1]]; }
            K++;
        } while(K<n); //K 从 1 到 n-1 循环
        S[I]=j; I=I+1; Sum=Sum+Dtemp;
    } while(I<n); //I 从 1 到 n-1 循环
    Sum=Sum+D[1][j];
    for(j=0;j<n;j++){ printf("%d",S[j]); } //输出城市编号序列
    printf("\n"); //换行
    printf("Total Length:%d",Sum); //输出总距离
}
```

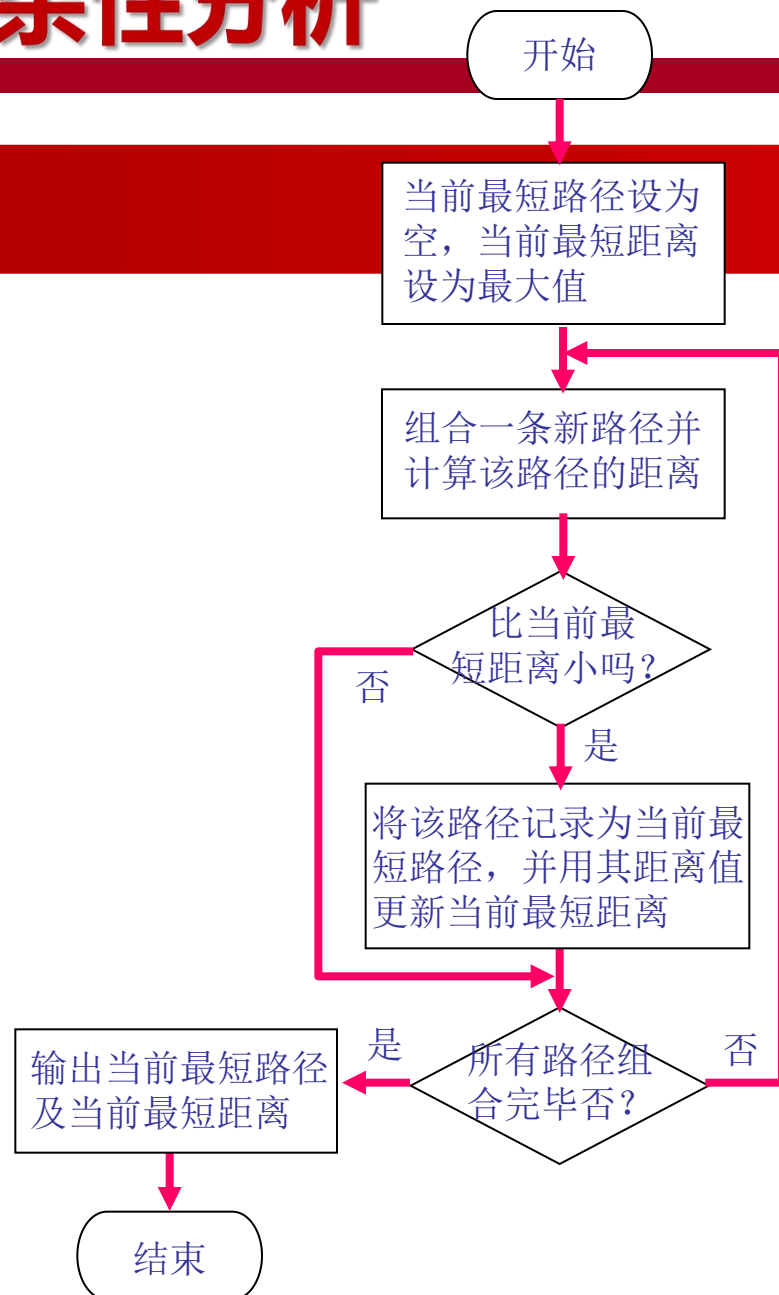
n

n²n³

TSP问题：算法的复杂性分析

TSP问题遍历算法的复杂性分析

- ✓ 列出每一条可供选择的路线，计算出每条路线的总里程，最后从中选出一条最短的路线。
- ✓ 路径组合数目为 $(n-1)!$
- ✓ 时间复杂度是 $O((n-1)!)$



TSP问题：算法复杂性与计算复杂性

TSP问题的计算复杂性与算法复杂性

- TSP问题的遍历算法的时间复杂度： $O((n-1)!)$
 - TSP问题的贪心算法的时间复杂度： $O(n^3)$ 。
 - TSP问题的计算复杂度： $O((n-1)!)$
- 算法复杂性**
-某一个算法的复杂性
- 计算复杂性**
-求精确解的那个算法的复杂性

TSP问题的遍历算法 (又称穷举或蛮干)

对解空间中的每一个可能解进行验证，直到所有的解都被验证是否正确，便能得到精确的结果---**精确解**

可能是 $O(n!)$
或 $O(a^n)$

TSP问题的 贪心算法

近似解求解算法---**近似解**

应该是 $O(n^a)$

$\Delta = | \text{近似解} - \text{精确解} |$
满意解： Δ 充分小时的近似解

排序问题求解案例

31

(1)从结构化数据表查找看排序问题的重要性

未排序

学号	姓名	成绩
120300101	李鹏	88
120300105	张伟	66
120300107	闫宁	95
120300102	王刚	79
120300103	李宁	94
120300106	徐月	85
120300108	杜岩	44
120300104	赵凯	69
120300109	江海	77
120300110	周峰	73

数据表记录数: n

查找成绩为80分的所有同学?

【算法A：未排序数据查找算法】

Start of algorithm

Step 1. 从数据表的第1条记录开始，直到其最后一条记录为止，读取每一条记录，做**Step 2**。

Step 2. 对每一条记录，判断成绩是否等于给定的分数：如果是，则输出；如果不是，则不输出。

End of algorithm

算法效率：读取并处理所有记录，即 n 条记录

排序问题求解案例

32

(1)从结构化数据表查找看排序问题的重要性

查找成绩为80分的所有同学？

已排序

学号	姓名	成绩
120300107	闫宁	95
120300103	李宁	94
120300101	李鹏	88
120300106	徐月	85
120300102	王刚	79
120300109	江海	77
120300110	周峰	73
120300104	赵凯	69
120300105	张伟	66
120300108	杜岩	44

数据表记录数: n

【算法B：已排序数据查找算法】

Start of algorithm

Step 1. 从数据表的第1条记录开始，直到其最后一条记录为止，读取每一条记录，做**Step 2**和**Step 3**步。

Step 2. 对每一条记录，判断成绩是否等于给定的分数：如果等于，则输出；如果不等于，则不输出。

Step 3. 判断该条记录的成绩是否小于给定的分数：如果不是，则继续；否则，退出循环，算法结束。

End of algorithm

算法效率：读取并处理部分记录，即 $\leq n$ 条记录

排序问题求解案例

33

(1)从结构化数据表查找看排序问题的重要性

已排序

学号	姓名	成绩
120300107	闫宁	95
120300103	李宁	94
120300101	李鹏	88
120300106	徐月	85
120300102	王刚	79
120300109	江海	77
120300110	周峰	73
120300104	赵凯	69
120300105	张伟	66
120300108	杜岩	44

数据表记录数: n

查找成绩为80分的所有同学?

【算法C：已排序数据查找算法】

Start of algorithm

Step 1. 假设数据表的最大记录数是 n ，待查询区间的起始记录位置**Start**为1，终止记录位置**Finish**为 n ；

Step 2. 计算中间记录位置 $I = (\text{Start} + \text{Finish}) / 2$ ，读取第 I 条记录。

Step 3. 判断第 I 条记录成绩是否大于给定查找分数：

(1)如果是小于，调整 $\text{Finish} = I - 1$ ，如果 $\text{Start} > \text{Finish}$ 则结束，否则继续做Step 2；(2)如果是大于，调整 $\text{Start} = I + 1$ ，如果 $\text{Start} > \text{Finish}$ 则结束，否则继续做Step 2；(3)如果是等于，则输出，继续读取 I 周围所有的成绩与给定查找条件相等的记录并输出，直到所有相等记录查询输出完毕则算法结束。

End of algorithm

• 算法效率：除**极端情况**外读取并处理 $\leq n/2$ 条记录

排序问题求解案例

34

(2)从结构化数据表的统计看排序问题的重要性

学号	姓名	成绩
120300107	闫宁	95
120300103	李宁	94
120300101	李鹏	88
120300106	徐月	85
120300102	王刚	79
120300109	江海	77
120300110	周峰	73
120300104	赵凯	69
120300105	张伟	66
120300108	杜岩	44

???

- 统计各个分数段的人数
- 统计每个同学的平均分数
- 统计每门课的平均分数

学号	姓名	成绩
120300101	李鹏	88
120300105	张伟	66
120300107	闫宁	95
120300102	王刚	79
120300103	李宁	94
120300106	徐月	85
120300108	杜岩	44
120300104	赵凯	69
120300109	江海	77
120300110	周峰	73

- 算法效率：需要读取并处理???条记录才能完成呢？

排序问题求解案例

35

(3)排序算法的重要性

学号	姓名	成绩
120300107	闫宁	88
120300103	李宁	66
120300101	李鹏	95
120300106	徐月	79
120300102	王刚	94
120300109	江海	85
120300110	周峰	44
120300104	赵凯	69
120300105	张伟	77
120300108	杜岩	73

排序算法是计算学科中很重要的算法

当对数据集需要多遍处理时，先排序-好

排序算法是很多复杂算法的基础

• 算法效率：需要读取并处理???条记录才能完成呢？

PageRank算法--网页排序问题

36

(1)网页排序问题：搜索引擎？



4,540,000条检索记录



1,210,000条检索记录

页面**链接**排序
优先？

网页**访问量**排
序优先？

怎样把最重要的检索记录显示给用户？

PageRank算法--网页排序问题

37

(2)PageRank是什么？网页又是什么？

问题背景--网页

●PageRank是计算
网页重要度的一种
方法



网页重要吗?---网页重要度

<标记>文本</标记>

Our Product Information

Our Product Information

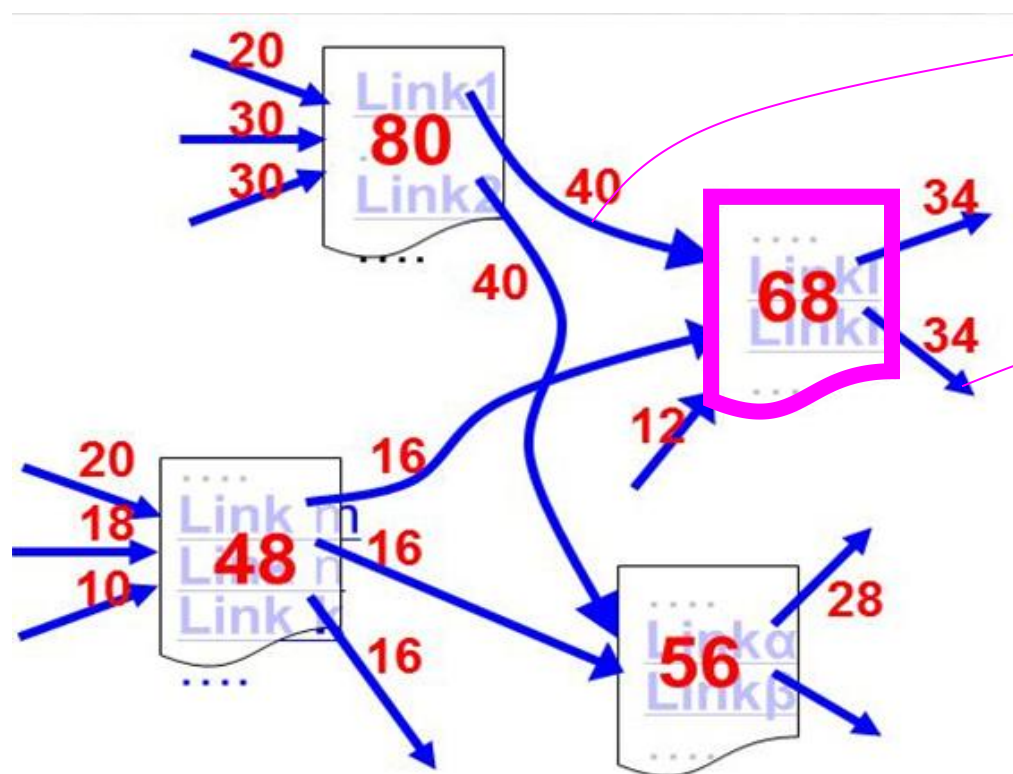
PageRank算法--网页排序问题

38

(3)正向链接与反向链接?

网页重要度问题的抽象

基于这些信息，
如何计算网页
重要度？



反向链接

≈高被引论文

正向链接

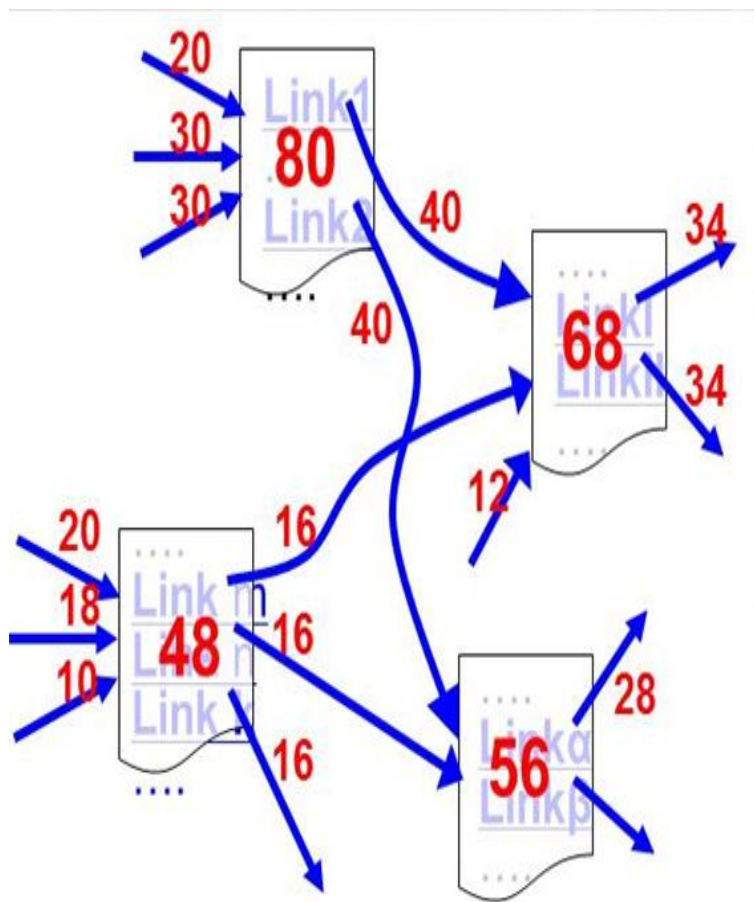
≈引用论文

一个网页的正向链接是另一个网页的反向链接

PageRank算法--网页排序问题

39

(4)PageRank的基本思想?



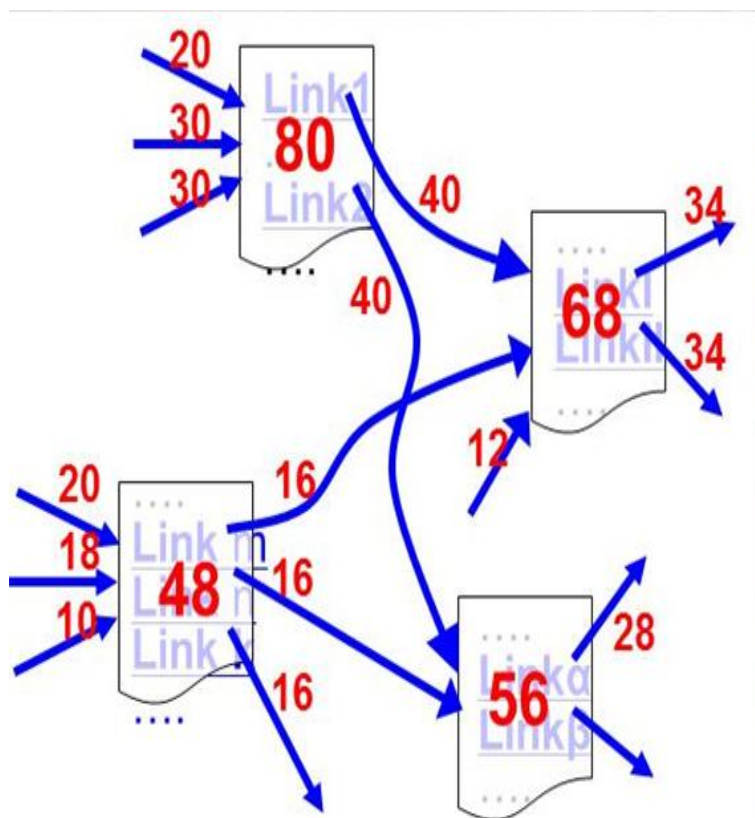
关于网页的基本观

- 网页的反向链接数越多是否越重要呢?
- 重要度越高的反向链接是否越重要呢?
- 正向链接数越多, 是否其对链接的网页而言, 重要度会降低呢?

PageRank算法--网页排序问题

40

(4)PageRank的基本思想?



- 一个网页的重要度等于其所有反向链接的加权和，即：反向链接权值 z_i ，网页重要度 R ，则

$$R = \sum z_i \text{ (for 所有反向链接 } i \text{)}。$$

- 一个正向链接的权值等于网页的重要度除以其正向链接数，即：网页重要度 R ，其正向链接数 m ，则其每一个正向链接的权值

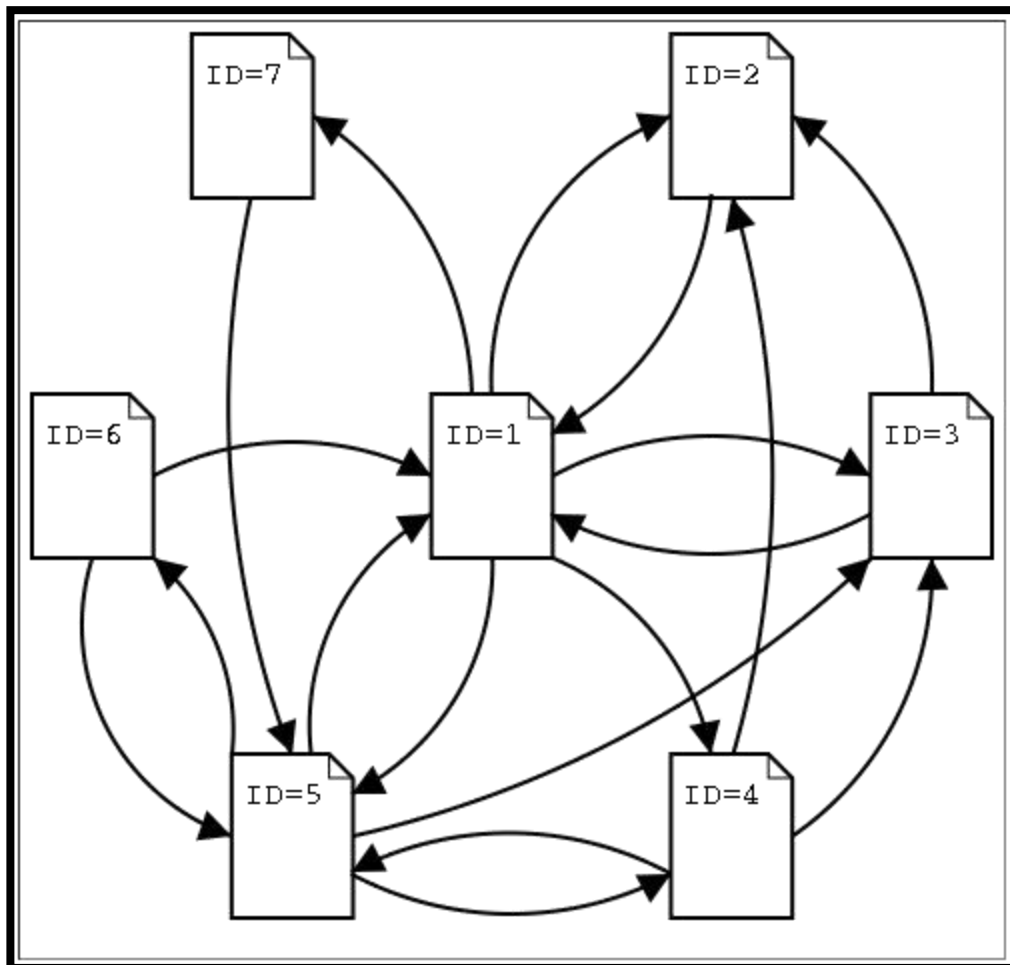
怎样计算网页的重要度呢？

$$z = R/m。$$

PageRank算法--网页排序问题表达与建模

41

(1)问题的数学建模：抽象



正向链接

链接源页面 ID	链接目标页面 ID
1	2, 3, 4, 5, 7
2	1
3	1, 2
4	2, 3, 5
5	1, 3, 4, 6
6	1, 5
7	5

PageRank算法--网页排序问题表达与建模

42

(2)问题的数学建模：邻接矩阵

行*i*, 列*j* 均是网页编号

$$a_{ij} = \begin{cases} 1 & \text{if (网页 } i \text{ 存在有指向网页 } j \text{ 的链接)} \\ 0 & \text{if (网页 } i \text{ 没有指向网页 } j \text{ 的链接)} \end{cases}$$

正向链接

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

反向链接

反向链接

$$A^T = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

正向链接

链接源页面 ID	链接目标页面 ID
1	2,3,4,5,7
2	1
3	1,2
4	2,3,5
5	1,3,4,6
6	1,5
7	5

PageRank算法--网页排序问题表达与建模

43

(3)正向链接的权值矩阵---转移概率?

反向链接

$$A^T = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

邻接矩阵

一个正向链接的权值等于网页的重要度除以其正向链接数

反向链接的权值

$$M = \begin{bmatrix} 0 & 1 & 1/2 & 0 & 1/4 & 1/2 & 0 \\ 1/5 & 0 & 1/2 & 1/3 & 0 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 0 & 1/2 & 1 \\ 0 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

转移概率矩阵

$$R^T = \begin{bmatrix} R_1 \\ R_2 \\ R_3 \\ R_4 \\ R_5 \\ R_6 \\ R_7 \end{bmatrix}$$

网页重要度向量

PageRank算法--网页排序问题表达与建模

44

(3)矩阵乘法与反向链接的加权和?

网页*i*的里安反为 R_i , 各网页里安反的向量 R , 记为: $R = (R_1, R_2, \dots, R_n)^T$

转移概率矩阵M

第n-1次
的网页重
要度

第n次
的网页
重要度

$$\begin{bmatrix} 0 & 1 & 1/2 & 0 & 1/4 & 1/2 & 0 \\ 1/5 & 0 & 1/2 & 1/3 & 0 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 0 & 1/2 & 1 \\ 0 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} R_1 \\ R_2 \\ R_3 \\ R_4 \\ R_5 \\ R_6 \\ R_7 \end{bmatrix}^{(n-1)} = \begin{bmatrix} R_1 \\ R_2 \\ R_3 \\ R_4 \\ R_5 \\ R_6 \\ R_7 \end{bmatrix}^{(n)}$$

矩阵乘法

$$R_i^{(n)} = \sum_j (M[i][j] * R_j^{(n-1)})$$

PageRank算法--网页重要度的迭代计算

45

(1)网页重要度的迭代计算方法?

网页*i*的重要度为 R_i ，各网页重要度的向量 R ，记为： $R = (R_1, R_2, \dots, R_n)^T$

迭代计算

$$R_i^{(1)} = \sum_j (M[i][j] * R_j^{(0)})$$

$$R_i^{(2)} = \sum_j (M[i][j] * R_j^{(1)})$$

... ..

$$R_i^{(n)} = \sum_j (M[i][j] * R_j^{(n-1)})$$

$$R_i^{(n)} = R_i^{(n-1)} \quad ???$$

转移概率矩阵M

$\begin{bmatrix} 0 & 1 & 1/2 & 0 & 1/4 & 1/2 & 0 \\ 1/5 & 0 & 1/2 & 1/3 & 0 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 0 & 1/2 & 1 \\ 0 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$	$\times$	$\begin{bmatrix} R_1^{(n-1)} \\ R_2^{(n-1)} \\ R_3^{(n-1)} \\ R_4^{(n-1)} \\ R_5^{(n-1)} \\ R_6^{(n-1)} \\ R_7^{(n-1)} \end{bmatrix}$	$=$	$\begin{bmatrix} R_1^{(n)} \\ R_2^{(n)} \\ R_3^{(n)} \\ R_4^{(n)} \\ R_5^{(n)} \\ R_6^{(n)} \\ R_7^{(n)} \end{bmatrix}$
---	----------	---	-----	---

第n-1次
的网页重
要度

第n次
的网页
重要度

矩阵乘法

R的初始值是多少呢？从哪一个 R_i 开始计算呢？

PageRank算法--网页重要度的迭代计算

46

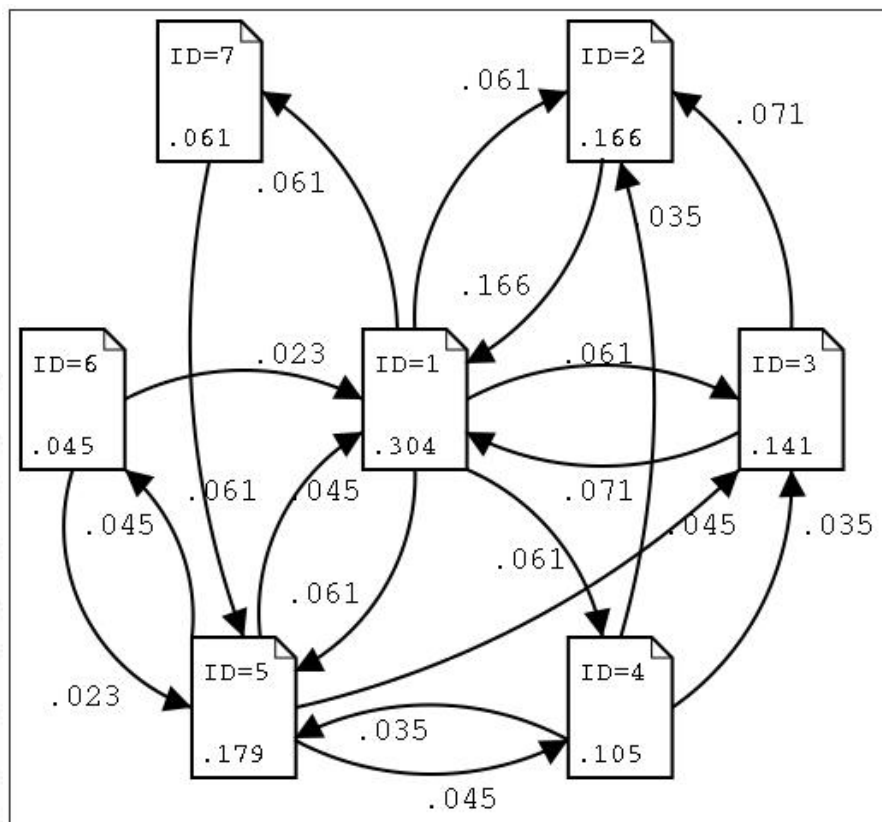
(2)PageRank的计算结果分析?

1号 vs. 5号

2号 vs. 3号

6号 vs. 7号

名次	PageRank	所评价的文件 ID	发出链接 ID (正向链接)	被链接 ID (反向链接)
1	0.304	1	2,3,4,5,7	2,3,5,6
2	0.179	5	1,3,4,6	1,4,6,7
3	0.166	2	1	1,3,4
4	0.141	3	1,2	1,4,5
5	0.105	4	2,3,5	1,5
6	0.061	7	5	1
7	0.045	6	1,5	5



PageRank算法--PageRank与数学及算法总结

47

(1)PageRank计算 vs. 数学的特征方程?

网页重要度的迭代计算

$$\begin{bmatrix} 0 & 1 & 1/2 & 0 & 1/4 & 1/2 & 0 \\ 1/5 & 0 & 1/2 & 1/3 & 0 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 0 & 1/2 & 1 \\ 0 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} R_1 \\ R_2 \\ R_3 \\ R_4 \\ R_5 \\ R_6 \\ R_7 \end{bmatrix}^{(n-1)} = \begin{bmatrix} R_1 \\ R_2 \\ R_3 \\ R_4 \\ R_5 \\ R_6 \\ R_7 \end{bmatrix}^{(n)}$$

迭代计算网页重要度

$$R^{(0)} = (R_1^{(0)}, R_2^{(0)}, \dots, R_n^{(0)})^T$$

$$R^{(1)} = cMR^{(0)}$$

$$R^{(2)} = cMR^{(1)}$$

...

$R = R^{(n)} = R^{(n-1)}$, 当R不发生变化时, 即收敛时则为所求

$$MR = \frac{1}{c} R$$

对 N 阶方阵A(转移概率矩阵), 满足:

$$Ax = \lambda x$$

的数 λ 称为 A 的特征值, 称 x 为属于 λ 的特征向量。

通过数学学习求解方法

PageRank算法--PageRank与数学及算法总结

48

数学的语义：特征方程

求解思想：求稳定性

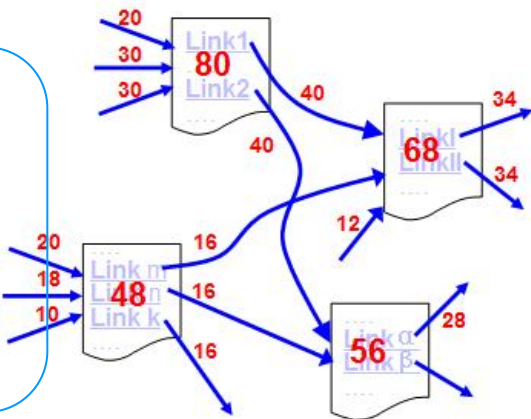
表达成数学：0,1矩阵
权值矩阵—转移概率矩阵

从问题语义挖掘求解思想：
反向链接数越多越重要；
反向链接有权值；
反向链接的权值确定：网页
重要度按其正向链接的个数
进行分配。

网页链接：正向链接与反向链接

网页排序：网
页重要度计算

$$Ax = \lambda x$$



网页重要度计算：PageRank

网页 i 的重要度为 R_i ，各网页重要度的向量 R ，记为：

$$R = (R_1, R_2, \dots, R_n)^T$$

需要迭代计算，第 j 次迭代计算得到的 R 的结果记为 $R^{(j)}$ 。

R 的初始可设置为任意的值，记为： $R^{(0)} = (R_1^{(0)}, R_2^{(0)}, \dots, R_n^{(0)})^T$

$$R^{(1)} = cMR^{(0)}$$

$$R^{(2)} = cMR^{(1)}$$

... ..

$$R^{(n)} = cMR^{(n-1)}$$

$R = R^{(n)} = R^{(n-1)}$ ----收敛状态暨稳定状态

$$M = \begin{bmatrix} 0 & 1 & 1/2 & 0 & 1/4 & 1/2 & 0 \\ 1/5 & 0 & 1/2 & 1/3 & 0 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 1/3 & 0 & 1/2 & 1 \\ 0 & 0 & 0 & 0 & 1/4 & 0 & 0 \\ 1/5 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \text{PageRank} = \begin{bmatrix} 0.303514 \\ 0.166134 \\ 0.140575 \\ 0.105431 \\ 0.178914 \\ 0.044728 \\ 0.060703 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}, \quad A^T = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$